Execution-Aware Gradient Supervision for Partially Observed Robot Dynamics

EAGM Research Draft — author list to be finalized

November 2026; exploratory analyses explicitly separated

Abstract

Robot world models receive policy commands, while physical transitions are driven by commands transformed by execution software. Hidden controller memory can therefore affect both prediction and the gradients used for policy optimization. We study EAGM, which combines a structured observable-command bottleneck with finite-difference policy-gradient supervision. In a fixed, partially unknown execution-controller setting, independent five-seed experiments support improved physical tracking returns over a matched model without directional supervision, including a simulator-budget control. These benefits do not establish general robot-task transfer. A true end-effector reaching evaluation fails to reproduce the benefit. Frozen-model interventions show that correcting reward geometry alone is insufficient, whereas replacing transition derivatives restores gradient alignment. Subsequent optimizer and start-distribution checks nevertheless fail joint held-out improvement criteria. The results distinguish prediction accuracy, gradient accuracy, and physical update utility, and identify the scope and remaining limitations of execution-aware supervision.

1 Introduction

A robot command is not necessarily the actuator input that causes the next observed transition. Control software may accumulate target positions, filter incoming commands, buffer actions, or limit command rates. Some of this execution state is observable; other parameters and memories may be unavailable to a deployed predictor. This interface matters when a world model is differentiated to improve a policy, rather than used only to predict a state.

A low prediction error does not constrain all local derivatives. Moreover, even an accurate policy gradient is defined for a particular objective and starting-state distribution. The optimizer then transforms that gradient into an update. Our central question is consequently restricted: can command-structured modeling and simulator-derived gradient supervision improve physical policy returns under partially observed execution dynamics, and where does this approach fail?

EAGM predicts an executed command using a nominal observable update plus a learned residual. A dynamics network maps the physical state and predicted command to the next state. Directional supervision compares a differentiated imagined return with finite differences of a physical rollout at the same policy parameters. The implementation uses a masked, coordinate-normalized Huber objective; it is not a pure cosine loss.

The evidence has three distinct levels. First, paired tracking experiments and independent confirmation seeds support return improvements within the tested controller setting. Second, budget controls and architectural comparisons constrain explanations based solely on extra interaction or an unconstrained auxiliary head. Third, reaching experiments expose a failure boundary and enable frozen-gradient interventions. These interventions are diagnostic privileges, not deployable algorithms.

Our contribution claim is the evaluated combination of an execution-command bottleneck and query-accounted gradient supervision in this restricted setting, together with an explicit analysis of its limitations. We do not claim that gradient-aware model learning is new, that EAGM achieves public-benchmark state of the art, or that an oracle establishes a successful new method. The submitted execution-memory diagnostic is a separate project; the overlap and additional algorithmic evidence must be documented before submission of this manuscript.

2 Related Work and Novelty Boundary

Policy-aware model fitting. PAML explicitly seeks agreement between true and model-derived policy gradients [1]. Its population discrepancy in Eqs. (7)–(8) compares policy-gradient functionals, and its empirical construction uses real and model trajectories with a critic estimated from real transitions. Thus, direct interest in policy-gradient quality predates EAGM. Our resettable finite-difference target and executed-command bottleneck instantiate a different experimental interface; they do not by themselves establish novelty over all PAML variants.

GAMPS learns a model through a weighted transition-likelihood objective [2]. Its Eq. (3) uses weights combining discounting, trajectory importance ratios, and cumulative policy-score norms. In schematic notation,

$$\max_{\hat{p}} \sum_{i,t} w_t^i \log \hat{p}(s_{t+1}^i \mid s_t^i, a_t^i). \tag{1}$$

This differs from EAGM's coordinate-level discrepancy between a differentiated imagined return and resettable-simulator finite differences. A fair empirical comparison still requires matching policy class, command semantics, observations, reset access, and interaction cost; no such external-baseline result is currently available.

Value and gradient propagation. VaGraM weights model errors using value gradients [5], emphasizing that prediction objectives should reflect downstream decisions. Action-conditioned world models (AWMs) address the path through which policy gradients propagate, rather than merely changing a prediction loss [4]. Their results make architecture-based gradient routing a serious alternative explanation and baseline. We use AWM to mean the action-conditioned models of Ma et al., not unrelated algorithms sharing the acronym.

Execution interfaces and differentiable control. *Mind the Execution Gap* studies action-semantic mismatch in asynchronous world-model control and proposes interfaces for future action sequences and applied-action attribution [6]. This overlaps with the broad motivation of execution consistency. EAGM's proposed distinction is supervised gradient quality under partially unknown execution dynamics, not the discovery that execution interfaces matter.

SuperMimic uses a differentiable learned model and short imagined backpropagation for humanoid tracking [3]. Its controller inputs and code-level actuator assumptions have not yet been fully audited here; we therefore make no claim that it lacks execution information and no numerical comparison to its reported performance.

What remains unresolved. The present comparison identifies objective-level differences for PAML and GAMPS, not a completed novelty certification. Matching their objectives on the same command interface is an outstanding experiment. Full implementation review of recent execution-interface and humanoid methods is also pending. External results must never be inserted into EAGM tables as if evaluated on the same tasks or budgets. The cited sources are primary papers; the reference audit records which claims were inspected and which remain open.

3 Problem Formulation

Let $x_t \in \mathbb{R}^{28}$ contain the positions and velocities of fourteen controlled joints. The actor emits an incremental command $a_t \in \mathbb{R}^{14}$. Execution state $c_t = (c_t^{\text{obs}}, c_t^{\text{hid}})$ includes observable target/filter states and potentially hidden memory. With execution parameters ξ,

$$u_t = U(c_t, a_t; \xi), \tag{2}$$

$$c_{t+1} = C(c_t, a_t; \xi), \tag{3}$$

$$x_{t+1} = F(x_t, u_t). \tag{4}$$

The predictor does not receive hidden controller parameters as deployment inputs. Executed commands can be logged as training labels. This is distinct from assuming that all future executed commands are known during planning.

The actor π_θ uses the physical state, observed target and task reference. For starting snapshot z drawn from distribution μ, define

$$J_\mu(\theta) = \mathbb{E}_{z\sim\mu} \sum_{t=0}^{H-1} \gamma^t r(x_{t+1}, a_t, a_{t-1}; z). \tag{5}$$

A snapshot must specify execution memory as well as physical state. Restoring only joint positions does not define the same finite-difference experiment. In the simulator audit, integrator warm-start state is also retained; it is numerical context, not an additional deployable observation.

Three errors. Prediction error compares $\hat{x}_{t+1}$ to x_{t+1}. Gradient error compares $\nabla_\theta \hat{J}$ with a reference for J. Update utility evaluates $J(\theta+\Delta\theta) - J(\theta)$. These quantities are neither interchangeable nor necessarily monotone. Coordinate normalization and controller discontinuities further affect their interpretation.

For a locally differentiable augmented transition $z_{t+1} = f(z_t, \pi_\theta(z_t))$, the complete sensitivity obeys

$$S_{t+1} = (f_z + f_a \pi_z) S_t + f_a \pi_\theta, \qquad S_0 = 0. \tag{6}$$

The policy feedback term cannot be dropped when analyzing a closed-loop actor. A correct reward derivative can be distorted by inaccurate transition sensitivities before it reaches policy-parameter space.

A limited local statement. If the evaluation objective has an L-Lipschitz gradient along the proposed update segment, the smoothness inequality gives

$$J_\nu(\theta + d) - J_\nu(\theta) \geq \nabla J_\nu(\theta)^\top d - \tfrac{L}{2}\|d\|^2. \tag{7}$$

An accurate ∇J_μ does not ensure positive alignment with d under another distribution ν. An optimizer may also rotate the update relative to its input gradient. This inequality is a conditional explanatory tool, not a theorem of EAGM convergence: clipping and rate limits can violate smoothness, and we do not estimate L in the experiments.

4 EAGM Method

Structured executed-command bottleneck. The command model is

$$\hat{u}_t = U_0(c_t^{\text{obs}}, a_t) + \Delta_\omega(c_t^{\text{obs}}, a_t). \tag{8}$$

For the tested interface, U_0 adds an incremental action to the target, clips it to joint limits, and applies the nominal observable filter. The residual compensates for discrepancies left by this nominal map. It does not read hidden gain, extra-lag, rate-limit or smoothing parameters. The physical predictor is $\hat{x}_{t+1} = F_\phi(x_t, \hat{u}_t)$, implemented as a normalized state-increment network. Passing only the command bottleneck to this network distinguishes it from a general MLP with an auxiliary command head.

Targets at matched policy parameters. At a sampled policy vector θ^k and restored training snapshot, generate a physical finite-difference reference

$$g_j^{\text{FD}} = \frac{J(\theta^k + \epsilon e_j) - J(\theta^k - \epsilon e_j)}{2\epsilon}. \tag{9}$$

Boundary-aware protocols use feasible stencils and record one-sided modes separately. The model gradient $\hat{g} = \nabla_\theta \hat{J}(\theta^k)$ must be computed at the same policy vector. Direction labels are not reusable across arbitrarily changed rewards: the Reach correction regenerated them under the true end-effector objective.

Implemented training objective. Normalized dynamics and command losses are combined with

$$\mathcal{L} = \mathcal{L}_{\text{dyn}} + \lambda_u \mathcal{L}_{\text{cmd}} + \lambda_d \mathcal{L}_{\text{dir}}, \tag{10}$$

$$\mathcal{L}_{\text{dir}} = \frac{\sum_j m_j \, h_1\left(\frac{\hat{g}_j - g_j^{\text{FD}}}{\max(|g_j^{\text{FD}}|, \tau)}\right)}{\sum_j m_j}, \tag{11}$$

where h_1 is the unit-threshold Huber loss and m_j indicates finite-difference numerical stability. The scale floor is $\tau = \max(0.1 s_0, 10^{-8})$, with stored label scale s_0. This objective supervises normalized gradient discrepancies, including magnitude, rather than pure signs or a cosine. If no coordinate is stable, the term is skipped. Boundary and numerical masks have different meanings.

The retained implementation applies a label every ten world-model updates and differentiates through the policy gradient to train the model. The paired trainer shares initialization and minibatches between Base and Dir. It uses $\lambda_u = 1$ and $\lambda_d = 0.1$ in the frozen protocol. Base omits $\mathcal{L}_{\text{dir}}$. Dir-Random retains randomly sampled policy probes; path-conditioned probes did not yield reliable gains.

Deployment and cost. Finite-difference labels require resettable simulator access and consume physical control steps. They are not free labels or online hardware gradients. Deployment uses the learned command/dynamics model, not the full-state Jacobian oracle used in later diagnostics. The current evidence does not establish how to obtain comparable supervision safely on a physical robot.

5 Experimental Setup and Evidence Protocol

Platform and tasks. Experiments use a bimanual OpenArm MuJoCo environment with fourteen actions and a 28-dimensional physical state. Controller B introduces partially unknown gain, extra lag, rate limiting and smoothing. Observable execution state is provided consistently, while hidden parameters are unavailable to deployable predictors. Tracking is the main method-evaluation task. Reach changes the objective to two-arm end-effector position control with a one-time inverse-kinematics reference; it introduces neither visual observations nor contact manipulation.

Models and budgets. Main comparisons include a memory-conditioned baseline Mc, EAGM-Base, EAGM-Dir, and ordinary/auxiliary-command MLP controls where available. The fixed main protocol uses 3000 world-model updates and 50 policy updates. The budget control reallocates direction-label simulator steps to additional dynamics data for Base. Exact stage-specific configurations and source CSVs are frozen; experiments with differing controllers or pools are never merged into a larger nominal sample.

Splits and pairing. Trajectory-level train/validation/test isolation replaces earlier transition-level splitting. Base/Dir pairing requires the same initialization, data split and minibatch stream. A later audit found different Base versus Dir minibatch streams in Stage6C; that contrast is exploratory, whereas Random versus Path retains its relevant pairing. Superseded Stage3 protocols and surrogate-only Reach results are not main performance evidence.

Physical reward and numerical references. Reach primary evaluation uses current MuJoCo end-effector positions, with separate scratch geometry to avoid stale post-integration site caches. Its reward is

$$r = -\tfrac{1}{6}\|p_{\mathrm{EE}} - p^*\|^2 - \tfrac{0.0005}{14}\|\dot{q}\|^2 - \tfrac{0.05}{14}\|a_t - a_{t-1}\|^2. \quad (12)$$

The velocity term is joint velocity, not end-effector velocity. Discount is 0.99. H8 gradient diagnostics and H16 physical returns are reported separately. Surrogate-scored physical rollouts are diagnostic only.

Statistics and evidence classes. Within-seed, paired returns are aggregated before seed-level statistics. We report seed wins and percentile bootstrap intervals, plus cluster and leave-one-seed sensitivity. Five seeds are few: a two-sided exact sign-flip test cannot achieve $p < 0.05$ with five observations even when every difference is positive. Bootstrap intervals excluding zero do not remove this limitation or account for historical experiment selection.

We distinguish repeated policy-return evidence within the tested setting; exploratory gradient/mechanism evidence; and failed task generalization. Starts within one frozen checkpoint are not independent model seeds. The new optimizer audit preregisters its start selection and update norm before collecting two fresh test episodes; even those episodes are only an exploratory evaluation, not confirmation.

6 Main Tracking Results

The strongest positive evidence remains the fixed Controller B tracking experiments. Table 1 is generated from the independently recomputed historical statistics, without pooling stages. Its provenance file records the raw per-start CSVs and hashes.

Comparison	Mean	95% CI	Wins
4: Dir–Mc	6.75	[1.24, 14.58]	4/5
4: Dir–Base	10.06	[5.77, 13.03]	5/5
5A: Dir–Mc	8.60	[4.72, 12.20]	5/5
5A: Dir–Base	5.14	[2.42, 8.16]	5/5
5B: budget	2.59	[2.11, 3.07]	5/5

Table 1: Paired return differences in units of 10^{-4}. CIs are within-stage seed bootstrap intervals; no cross-stage pooling.

Direction supervision improves over the matched Base model in all five Stage4 seeds and in all five independent Stage5A confirmation seeds. The Stage5A mean difference is 5.14×10^{-4} in physical return. Against Mc, Stage4 improves in four of five seeds, while Stage5A improves in five of five. The effect sizes and seed variability are more informative than calling all of these comparisons statistically significant.

The Stage5B budget comparison assigns the finite-difference control-step allowance to additional dynamics training data for Base. Dir still improves in five of five seeds, with mean difference 2.59×10^{-4}. This rules out one specific extra-data explanation under that protocol; it does not establish equality of wall-clock compute or all possible uses of the simulator budget.

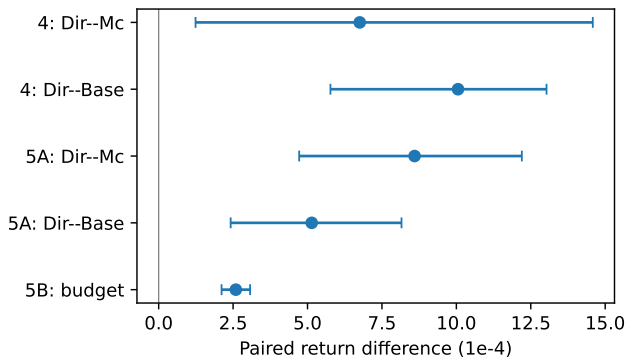


Figure: Mean paired physical-return differences and seed-bootstrap intervals. Stage4, Stage5A and Stage5B remain separate experiments.

Architectural controls provide additional evidence for the command bottleneck: the Stage5C Base comparison improves over the auxiliary-command and observable-memory MLP controls in five of five seeds. These controls should be described through their actual input and parameter budgets, rather than called generic state-of-the-art baselines. The final submission still needs an externally motivated objective baseline such as a matched PAML/GAMPS adaptation.

We restrict the main claim to the evaluated tracking distributions. Later independent controller-randomization seeds support Dir over Base, but comparisons with Mc remain uncertain. Reach does not support the same performance claim. Neither diagnostic oracle outcomes nor multiple starts from the same checkpoint enlarge the sample size of this main table.

7 Ablations, Distribution Shift, and Failure Boundaries

Controller shift. Controller-randomized interpolation and extrapolation experiments provide positive Dir–Base differences across the five independent Stage6B.1 seeds. Against Mc, however, uncertainty remains: extrapolation and interpolation bootstrap intervals can include zero. We therefore distinguish improvement over an architecture-matched Base from superiority over a stronger memory baseline. Neither establishes arbitrary-controller robustness.

Rejected extensions. History alone can improve some dynamics prediction metrics without a matching return improvement. HistC, which adds observed control-state history, did not show a reliable independent temporal-structure benefit against capacity and ordering controls. Path-Dir did not consistently improve over Dir-Random; only two of five seed differences were positive. Reduced policy learning rates removed parameter saturation without producing a reliable performance gain. These are reasons to retain a smaller supported method, not modules to be repackaged as contributions.

Reach protocol correction. The initial reaching experiments used a learned FK surrogate for both imagined reward and the reported physical finite-difference objective. A surrogate-path parity check could not establish agreement with the actual end-effector objective. The corrected experiment regenerated direction labels using true MuJoCo geometry and retrained Dir. On seed20, Dir's mean true return was approximately -0.015714, versus Base -0.015449; Dir's change from initialization was negative. This is a failed transfer experiment, not evidence of task generalization.

Geometry decomposition. The geometry audit measured position RMSE around 2.24 cm, relative Jacobian error around 30%, and Jacobian cosine 0.964 on its held-out trajectory-state subset. These metrics use a different state distribution from the earlier FK-fit acceptance test and must not be substituted for it. Replacing the position residual while retaining the FK Jacobian improved state-level reward-gradient cosine from 0.441 to 0.957. Replacing the Jacobian alone did not help. Thus, a large relative derivative error did not imply a comparably large directional error.

Complete true-reward derivative replacement still left the mean policy-gradient cosine negative (about -0.116). It partially improved the 50-update physical return but did not restore the initial-policy return. This distinguishes state-level reward geometry from full policy-gradient propagation and motivates the next analysis; it does not support a Geometry-aware EAGM method.

Draft status: The supplement should include stage-specific OOD tables, all seed differences, capacity counts, and failed-module controls. Their archived raw results are retained; no new module or cross-task success is inferred.

8 Gradient Propagation and Update Utility

Frozen dynamics intervention. A local audit augments physical state with execution memory and solver warm-start, producing a 154-dimensional diagnostic state. The learned state Jacobian has mean cosine 0.073 on six starts. A conditional physical-dynamics probe using actual executed commands also finds poor command derivatives (cosine about0.079). The full oracle replaces both execution and physical-transition derivatives; their separate causal shares are not fully identified.

Holding forward states and reward values identical, the H8 policy-gradient cosines are:

Reward derivative	Learned f	Oracle f
FK	-0.132	0.795
True	-0.116	0.987

Local physical-chain differentiation agrees with whole-policy finite differences at cosine 0.999999 on the predefined implementation-check start. The oracle is privileged and diagnostic, not an implementable learned-policy algorithm. Its gradient recovery did not imply better held-out return after the budgeted one-step Adam update.

Optimizer–data alignment. A preregistered follow-up compares K2/K4 training starts and raw/first-Adam updates rescaled to the same L2 norm. Both gradient sources use true reward derivatives. It collects two new test episodes and retains two validation episodes independent of training; historical test starts remain exploratory.

Oracle update	Validation	Fresh test
K2 Raw	-5.410	-2.482
K2 Adam	+0.954	-1.255
K4 Raw	-14.261	+7.677
K4 Adam	-7.096	+4.264

Mean true-return changes in units of 10^{-5}. Two starts per column, one frozen checkpoint; no significance claim.

The true aggregate training-gradient cosine with validation changes from -0.290 at K2 to -0.830 at K4, while alignment with the fresh test changes from -0.177 to 0.381. Increasing coverage improves one sampled distribution while hurting another. Both preregistered joint screens fail. We do not select the apparent fresh-test winner as a method.

Limitations and conclusion. EAGM has multi-seed tracking evidence within a restricted execution setting, but no demonstrated public-benchmark SOTA result, visual contribution, or physical-robot validation. Reach remains a failure boundary. The diagnostic phases use one frozen checkpoint, small start sets, and privileged simulator access. Local smoothness, exact reset access and numerical finite differences are additional assumptions. Extra model parameters are not a substitute for independent validation.

The evidence supports execution-aware gradient supervision as a useful restricted approach and transition derivatives as one causal source of gradient distortion. It does not establish that accurate gradients guarantee useful updates across start distributions. A deployable method addressing that remaining gap requires new, independently validated evidence; the present oracle analyses do not supply it.

References

[1] Romina Abachi, Mohammad Ghavamzadeh, and Amir-massoud Farahmand. Policy-aware model learning for policy gradient methods. *arXiv:2003.00030*, 2021.

[2] Pierluca D'Oro, Alberto Maria Metelli, Andrea Tirinzoni, Matteo Papini, and Marcello Restelli. Gradient-aware model-based policy search. In *AAAI*, 2020.

[3] Qiayuan Liao, Yiyang Shao, Junfeng Long, Kevin Zakka, Roberto Horowitz, Pieter Abbeel, and Koushil Sreenath. Supermimic: Scaling humanoid whole-body control with one differentiable world model. *arXiv:2610.28480*, 2026.

[4] Michel Ma, Tianwei Ni, Clement Gehring, Pierluca D'Oro, and Pierre-Luc Bacon. Do transformer world models give better policy gradients? In *ICML*, pages 33855–33879, 2024.

[5] Claas Voelcker, Victor Liao, Animesh Garg, and Amir-massoud Farahmand. Value gradient weighted model-based reinforcement learning. *arXiv:2204.01464*, 2023.

[6] Shengtao Wen, Xiang Chen, Yu Tian, Lingbing Guo, Lina Gong, and Sheng-Jun Huang. Mind the execution gap: Action-semantic mismatch in world-model control. *arXiv:2610.06582*, 2026.